

Energy-Efficient Deep Learning Without Backpropagation: A Rigorous Hardware-Validated Benchmarking Study of Forward-Only Algorithms

Przemysław Spyra¹[0009–0009–3001–2604] and Witold
Dzwiniel¹[0000–0001–8321–5928]

AGH University of Krakow, Faculty of Computer Science
przspyra@agh.edu.pl, dzwiniel@agh.edu.pl

Abstract. Backpropagation (BP) remains the dominant algorithm for training deep neural networks, yet its memory and energy costs motivate the search for practical alternatives. We present a hardware-validated benchmarking study of three forward-only, BP-free training algorithms — Forward-Forward (FF), Cascaded Forward (CaFo) and Mono-Forward (MF) — against tuned BP baselines on MLP and 3-block CNN architectures across four standard vision benchmarks, with energy, time and memory read directly from the device through the NVIDIA Management Library (NVML). Two results follow from enforcing a single stopping rule across algorithms. First, MF matches or surpasses BP in accuracy on its native MLP architectures, by 0.01 to 5.04 pp, and holds up to 27% less per-process device memory; but it is not cheaper to train. An earlier reading of these experiments reported 41% less energy and 34% less time for MF; that comparison charged the two algorithms different amounts of training before either could stop, and equalising the rule reverses its sign. Second, an ablation ladder that interpolates between BP and MF one change at a time — adding a deep-supervision BP baseline, a learned readout, gradient locality and activation caching — attributes essentially the entire cost increase to gradient locality (228 to 272 pp of BP’s energy) and shows that deep supervision alone reproduces MF’s accuracy on the MNIST-scale tasks but not on CIFAR, where MF leads it by over 5 pp. All measurements, code and per-run traces are public.

Keywords: backpropagation-free learning · forward-only algorithms · energy efficiency · hardware benchmarking · deep neural networks

1 Introduction

Deep neural networks have achieved transformative success across AI [9], yet this progress rests on an escalating energy budget; by 2030 the operational requirements of such models are projected to consume between 3% and 4% of global electricity [20]. Individual training runs are already substantial: [19]

estimated roughly 284 t of CO₂e for a single large Transformer trained with neural architecture search, and [15] reported approximately 552 t for GPT-3.

Among the algorithmic contributors to that cost, backpropagation (BP) [18] is central. Its dependence on a full backward pass requires storing all intermediate activations, giving memory costs proportional to network depth, and its layer-locked structure imposes a “backward locking” bottleneck that constrains parallelisation [6]. This motivates alternatives built on local learning rules and layer-by-layer credit assignment. None of the algorithms studied here claims full biological plausibility [12]; we treat that lineage as motivating context rather than as a validated claim. We study three such algorithms: **Forward-Forward** (FF) [4], which learns from a contrastive “goodness” objective over positive and negative forward passes; **Cascaded Forward** (CaFo) [22], which attaches a local classifier to each of a cascade of neural blocks; and **Mono-Forward** (MF) [3], which gives every hidden layer a learnable projection matrix and a local cross-entropy loss.

A fair evaluation of these methods against BP on direct hardware-level metrics remains largely unexplored; existing comparisons often differ in architecture or tuning effort, which confounds the algorithm [2]. This paper closes that gap, and in doing so reaches a conclusion the earlier literature — including an earlier reading of our own experiments — did not: once the stopping rule is the same for every algorithm, none of the three forward-only methods is cheaper to train than BP. The interesting question is then *why*, which we answer by ablation rather than by attribution to the algorithm as a whole.

Contributions. We contribute (i) a comparative benchmarking framework mandating identical native architectures, an equal tuning budget for every method, a single stopping rule and direct hardware-level measurement via NVML, with every protocol asymmetry we found in our own earlier configuration disclosed and corrected (Section 3); (ii) a hardware-validated benchmark of MF against a tuned BP baseline, which matches or surpasses BP in accuracy on every tested MLP and holds materially less per-process device memory, but costs more time and energy under an equal stopping rule; (iii) an ablation ladder from BP to MF — deep supervision, learned readout, gradient locality, activation caching — that attributes the cost to gradient locality alone and separates MF’s accuracy advantage from the effect of localised objectives (Section 4.4); and (iv) an empirical re-evaluation of memory efficiency, showing that algorithm-specific overhead can offset the theoretical saving from omitting the backward pass, and that the device-wide memory reading commonly quoted is dominated by the CUDA context.

2 Background and Related Work

We state each algorithm’s update mechanism, because that mechanism — not the parameter count — determines the shape of the resulting GPU workload. **Forward-Forward** (FF) [4] replaces the forward-backward pair with two forward passes, on “positive” (real) and “negative” (contrastive) data; each layer locally

maximises a “goodness” score on the former and minimises it on the latter, so credit assignment never leaves the layer. This doubles the data volume per step and supervises only indirectly, the usual explanation for its slow convergence [4, 22]. **Cascaded Forward** (CaFo) [22] follows each of a cascade of blocks with a local auxiliary classifier, giving direct cross-entropy supervision at several depths and removing the need for negative samples; blocks are either randomly initialised and frozen (CaFo-Rand) or pretrained with Direct Feedback Alignment (CaFo-DFA), which pays an extra pretraining stage before any predictor is useful. **Mono-Forward** (MF) [3] equips each hidden layer with a learnable projection matrix M_i mapping activations to class-wise goodness scores, on which a local cross-entropy loss is computed; layer i and M_i are updated from that loss alone with the incoming activation *detached*, so no gradient crosses a layer boundary and the layers train in sequence. That detachment is the property Section 4.4 isolates: it is what permits activation caching, and it is also what costs.

Forward-only and local-loss methods. FF has been extended with convolutional channel-wise competitive learning [14] and with collaborative layer objectives [13]; both report accuracy gains over baseline FF but no hardware-level efficiency comparison of the kind conducted here. [17] showed that forward-gradient methods augmented with local auxiliary losses scale favourably, grounding MF’s local projections, and [5] analysed local loss optimisation in the infinite-width limit, showing that greedy layer-wise training can reduce the effective complexity of the learned representation — the basis of the regularisation hypothesis of Section 5.

Deep supervision. Attaching auxiliary classifiers to intermediate layers and training end-to-end is the deep-supervision construction of [11]. MF resembles it closely, differing only in that MF detaches the incoming activation and reads out through M_L . We exploit that near-identity in Section 4.4: a deep-supervision BP baseline is the control that separates the effect of localised objectives from the effect of forward-only training.

Hardware and energy. GPUs and TPUs [7] are designed around BP’s forward-backward structure, and [19] and [15] give the empirical grounding for its energy cost. Our contribution is evaluative rather than algorithmic: to our knowledge this is the first hardware-validated comparison of FF, CaFo and MF under one fairness framework, with a deep-supervision control.

3 Framework for Fair and Rigorous Comparison

Throughout, a *hardware-validated comparison* is one in which every efficiency claim is backed by a physical quantity read from the GPU itself while the workload runs — energy, elapsed time and memory occupancy — rather than by an analytical proxy such as a FLOP or parameter count.

Fairness Protocols.

1. **Native architectures.** Each algorithm runs on the architecture of its source publication: **FF** and **MF** on their MLPs, **CaFo** on its 3-block CNN. CIFAR inputs are flattened to 1-D vectors for MLP experiments, following the **MF** protocol [3].
2. **Identical BP baselines.** Each BP baseline is built from the same module list as its forward-only counterpart with the algorithm-specific components removed — goodness heads for **FF**, projection matrices for **MF**, block predictors for **CaFo**. A reviewer observed that this removal could itself account for the differences we report. It can, and Section 4.4 measures how much, by putting those components back one at a time on top of BP.
3. **Equal trial budget, algorithm-appropriate search spaces.** Optuna [1] tuned every algorithm and every BP baseline for every configuration under an identical trial budget and trial epoch cap, maximising validation accuracy. The spaces are not identical, because the algorithms do not share a parameter set, but each covers at least three decades of learning rate and weight decay. The spaces and selected values are in the repository.
4. **One stopping rule.** All algorithms stop on validation loss [16] with the same minimum improvement, patience and epoch cap (Table 1).
5. **Fixed random seeds.** The submitted version used three seeds (42, 123, 7). Every result here is re-measured over at least five, and the **BP**→**MF** ladder over twenty or more; seed counts are stated per row and per caption.

Disclosure: what the submitted version actually did. An audit of our own configuration files against the protocols above found three asymmetries, none of which any reviewer raised and all of which we correct here. *(i) The stopping rule was not consistent* (Table 1). **BP** stopped on validation loss, **FF** on validation accuracy with a 0.01 minimum improvement, and **CaFo** and **MF** on per-predictor and per-layer local losses with per-configuration patiences and caps. Because those caps apply per stage rather than per run, a nominal cap of 15 epochs on a four-stage **MF** network permits 60 epochs of training. Time and energy are integrals over training duration, so an inconsistent stopping rule does not perturb the efficiency comparison — it determines it. This is the single largest correction in this revision. *(ii) Pruning was asymmetric.* **BP**’s Optuna studies used a median pruner while the others ran unpruned, so **BP**’s nominal 50 trials included truncated ones and its effective search was the weaker. The asymmetry therefore *disfavoured* **BP**, i.e. it worked in the direction of the submitted paper’s conclusion. Pruning is now disabled for every algorithm, paid for by a uniform 20-epoch trial cap. *(iii) Three configurations were not independently searched.* The Optuna sampler was seeded from the global experiment seed, so two studies over the same space returned byte-identical values: the **BP** MNIST 4×2000 baseline shared its hyperparameters with **BP** MNIST 3×1000 , and both **CaFo** CIFAR-100 variants with their CIFAR-10 counterparts. The sampler seed is now derived from the study name, and all three were re-searched; the replacements differ by $2.9\times$, $2.2\times$ and $15.3\times$ in at least one value.

Table 1. The stopping rule as submitted, against the harmonised rule used throughout this revision. Ranges span the per-configuration values actually set. The epoch cap is per *unit*: an epoch for BP and FF, a layer for MF, a predictor for CaFo. Every value is transcribed from the configuration files, which are preserved in the repository.

Algorithm	Stopping metric	Dir.	Min. Δ	Patience	Epoch cap
<i>As submitted</i>					
BP	validation loss	min	0	5–40	100 / run
FF	validation <i>accuracy</i>	max	0.01	20	100 / run
CaFo	local loss, per predictor	min	0.0005–0.001	5–10	12–1271 / pred.
MF	local loss, per layer	min	0.0001	4–10	8–15 / layer
<i>Harmonised (this revision)</i>					
all	validation loss	min	0	20	100 / unit

Architecture, optimiser and implementation. All weights and projection matrices use Kaiming uniform initialisation. BP and BP-DS use AdamW; CaFo predictors and MF layers use Adam on a local cross-entropy loss. The MLPs are fully connected with ReLU, 2–4 hidden layers of 1000 or 2000 units. Each CaFo block is a 3×3 convolution, ReLU, 2×2 max-pooling and batch normalisation with 32, 128 and 512 output channels; the three block predictors are summed to form the prediction. FF uses a fixed goodness threshold $\theta = 2.0$ with peer normalisation (factor 0.03, momentum 0.9). Every algorithm is implemented in the same PyTorch codebase on the same `nn.Linear` and `nn.Conv2d` primitives, differing only in the loss construction and in where `detach()` is called, so no method benefits from a fused or hand-written kernel the others lack.

Experimental Setup. Experiments used MNIST [10], Fashion-MNIST [21], CIFAR-10 and CIFAR-100 [8] on NVIDIA A100-SXM4-40GB GPUs (Athena cluster, ACK Cyfronet AGH) hosted on AMD EPYC 7742 nodes, with driver 595.71.05, Python 3.10.4 and PyTorch 2.4.0+cu121. The driver version is stated because it governs NVML power-reading semantics. For CIFAR-10 and CIFAR-100, random horizontal flips ($p=0.5$) and random 32×32 crops with 4-pixel padding were applied identically to every algorithm *and* its BP baseline, before flattening; MNIST and Fashion-MNIST were not augmented. The full environment listing, the Optuna search spaces and the exact NVML API calls are in the repository (Section 6).

Evaluation metrics and instruments. Accuracy is measured on the held-out test split. Efficiency is measured directly, and we name the instrument for every quantity because they are not interchangeable. A monitor thread queries the NVIDIA Management Library (NVML) — the driver’s own telemetry interface, chosen because it reports physical device state rather than an analytical proxy — at 5 Hz: power from `nvmlDeviceGetPowerUsage`, from which energy (Wh) is the trapezoidal integral over the run; utilisation from `nvmlDeviceGetUtilizationRates`; SM clock from `nvmlDeviceGetClockInfo`; device memory from `nvmlDeviceGetMemoryInfo`; and wall-clock time from the

host monotonic clock. Two memory figures exist and must not be conflated: the NVML reading is *device-wide* and includes a fixed ≈ 800 MiB CUDA context that no algorithm controls, whereas `torch.cuda.max_memory_allocated` is the per-process allocator high-water mark. We report the per-process figure and say so. Host resident set size, where quoted, comes from `psutil` and is a CPU-side quantity. Estimated CO₂e and GFLOPs are collected but reported only in the repository.

4 Experiments and Results

Under a stopping rule applied uniformly to every algorithm, none of the three forward-only methods is cheaper than BP; what separates them is how they spend the extra cost and what they buy with it. Results are mean $\pm$ SD over the seed counts stated per row.

4.1 FF: A Conceptually Sound but Computationally Costly Starting Point

FF trains to accuracy comparable with BP on its native MLP architectures, which validates its core premise, but it does so at a prohibitive efficiency cost (Table 2). On the Fashion-MNIST 4×2000 MLP it takes $7.4\times$ longer and draws $6.3\times$ the energy of the BP baseline; on MNIST 4×2000 the corresponding factors are $6.5\times$ and $5.9\times$. The accuracy advantage claimed in the submitted version does not survive re-measurement. The submitted FF numbers were produced by a checkpoint-selection defect that restored an epoch-2 network whenever the stopping metric was a loss rather than an accuracy; with that corrected and seven seeds per arm, FF is 0.70 pp *below* BP on Fashion-MNIST (CI = $[-0.87, -0.54]$) and indistinguishable from it on MNIST (-0.06 pp, $[-0.21, +0.07]$). The efficiency penalty is therefore not purchased by an accuracy margin; there is no margin.

The cost is not a matter of power draw: FF and BP sit at comparable wattage, and FF’s energy bill is run up almost entirely by how long it takes to reach its stopping criterion. Decomposing the end-to-end gap into epochs and cost per epoch makes this exact. FF needs 2.7 to $3.8\times$ more epochs than BP but spends only 1.5 to $2.0\times$ as much time and 1.5 to $1.8\times$ as much energy *per epoch*. Its disadvantage is convergence speed, not per-step inefficiency. The NVML traces rule out the obvious alternative explanations. Both algorithms hold the SM clock near its ceiling throughout (1323 MHz for BP, 1274 MHz for FF), so the penalty is not throttling; FF’s within-run clock variability is larger (SD 114 MHz against 67 MHz) but far too small to account for a $7.4\times$ gap. And on both instruments—NVML device-wide (296 MiB against 287 MiB) and the `torch` allocator—FF consumed slightly *more* peak memory than BP, refuting a key presumed advantage.

Table 2. Each algorithm on its native architecture against an identically configured, independently tuned BP baseline, under the harmonised stopping rule of Table 1. Mean $\pm$ SD over the stated seeds; the better result within each block is shown in **bold**. CIFAR-10 inputs are flattened to 3072-dimensional vectors for the MLP rows, following the MF protocol, and CaFo times include the CaFo-DFA block-pretraining stage. Time and Energy cover full training to early stopping; energy is the NVML power trace integrated over the run. Peak Mem is `torch.cuda.max_memory_allocated`, the per-process figure.

Algorithm	Accuracy (%)	Time (s)		Energy (W h)	Peak Mem (MiB)
<i>Fashion-MNIST 4×2000 MLP ($n = 7$)</i>					
FF-AdamW	88.90 ± 0.14	$486.0 \pm$	21.3	13.10 ± 0.59	295.9 ± 0.0
BP Baseline	89.61 ± 0.15	$65.6 \pm$	3.5	2.08 ± 0.16	287.1 ± 0.0
<i>CIFAR-10 3-block CNN ($n = 5$)</i>					
CaFo-Rand	66.75 ± 1.55	$889.9 \pm$	58.0	14.63 ± 0.99	122.8 ± 0.0
CaFo-DFA	78.43 ± 0.45	$1911.6 \pm$	112.7	38.48 ± 3.51	167.8 ± 0.0
BP Baseline	85.58 ± 0.40	$240.7 \pm$	23.8	4.63 ± 0.44	146.7 ± 0.0
<i>CIFAR-10 3×2000 MLP ($n = 20$)</i>					
MF	66.37 ± 0.28	$1002.1 \pm$	66.5	17.28 ± 1.12	216.8 ± 0.0
BP Baseline	62.47 ± 0.33	$253.0 \pm$	16.7	5.39 ± 0.39	299.0 ± 0.0

4.2 CaFo: A Structured Alternative with Acute Trade-offs

CaFo, on its native 3-block CNN, shows a stark accuracy–efficiency trade-off contingent on its feature-learning strategy—but neither variant is efficient. Re-measured with five seeds, CaFo-Rand is $3.7\times$ slower and draws $3.2\times$ the energy of BP on CIFAR-10 while losing 18.8 pp of accuracy, and CaFo-DFA recovers much of that accuracy only by spending $7.9\times$ the time and $8.3\times$ the energy (Table 2). Across all four datasets the time penalty ranges from $3.6\times$ to $10.5\times$ and the energy penalty from $2.9\times$ to $10.9\times$. The one genuine win is memory: CaFo-Rand holds 16% less peak per-process device memory than BP on CIFAR-10, because its fixed random extractors need no backward pass. As with FF, the per-epoch decomposition is more flattering than the total: CaFo-Rand is *cheaper* than BP per data pass (0.76 to $0.90\times$ the energy per epoch) and pays for it with 3.6 to $7.8\times$ more passes, whereas CaFo-DFA on CIFAR is expensive on both axes. These figures replace the back-ported values of the submitted version, which were measured on an earlier software stack and overstated CaFo’s standing considerably.

The CaFo-DFA power trace opens with an elevated ≈ 125 W plateau: the DFA block-pretraining stage, during which no predictor is yet useful for the downstream task. That cost is included in every reported CaFo-DFA time and energy figure. The design insight is that high-quality features are necessary for accuracy but that DFA is not an affordable way to obtain them.

4.3 MF: Higher Accuracy and Lower Memory, at a Higher Total Cost

Mono-Forward matches or surpasses BP accuracy on every tested MLP architecture, and the margin widens with task difficulty: +0.19 pp on MNIST, +0.01 pp on Fashion-MNIST, +3.90 pp on CIFAR-10 and +5.04 pp on CIFAR-100, each over twenty seeds (Table 2). It also holds materially less per-process device memory on the wider models—27% less on CIFAR-10 and 25% less on CIFAR-100—which is the structural consequence of never storing a global activation graph. Note that CIFAR-10 accuracy for MLP architectures is substantially lower than CNN-based state-of-the-art owing to the elimination of spatial structure via flattening; the comparison here is strictly between an MLP-based MF and an MLP-based BP baseline on identical architectures.

What does not hold is the efficiency claim. Under the harmonised rule MF is 4.0× slower than BP and draws 3.2× the energy on CIFAR-10, with the same pattern on all four configurations. The submitted version reported the opposite — MF 33.81% faster and 40.78% less energy-hungry — and that result is reproducible, but only under the legacy protocol, in which MF’s patience is spent once per layer while BP’s is spent once for the whole network. Re-running it on the current cluster over three seeds recovers 33.6% [30.2, 36.0] and 42.4% [38.9, 44.8], so the discrepancy is not a measurement error; it is that the comparison charged the two algorithms different amounts of training before either was allowed to stop. Equalising the rule reverses the sign. Section 4.4 isolates which architectural choice is responsible.

Hardware monitoring on the CIFAR-10 task shows what MF’s local training regime actually does to the device (Figure 1). MF draws less power and keeps the GPU less than half as busy as BP (7.3 % against 16.2 % mean utilisation), but it stays resident almost four times as long, so the lower instantaneous draw does not translate into lower energy. Its per-layer stages are visible as steps in both the utilisation and the memory trace.

Across all 20 seeds of every rung, total energy and training time are collinear along a single iso-power band: the rungs differ in how long they run, not in how hard they drive the device. MF nonetheless reaches higher accuracy than BP on identical architectures; Section 5 discusses a regularisation hypothesis. We do not compare validation-loss curves, because MF optimises a separate local objective per layer and logs no global validation series.

4.4 Where the Cost Comes From

Attributing MF’s cost to MF as a whole explains nothing. We therefore interpolate between BP and MF in single-change steps—auxiliary supervision, the learned readout M_L , gradient locality, and, where activations can be cached, two caching strategies—and measure each step separately (Figure 2). One step dominates. Detaching the gradient at every layer boundary adds between 228 and 272 pp of BP’s energy across the four configurations, while auxiliary supervision and the readout together move it by at most 40 pp and on the two CIFAR tasks partly cancel. The overhead is not an implementation detail that tuning will remove: it is the price of locality itself.

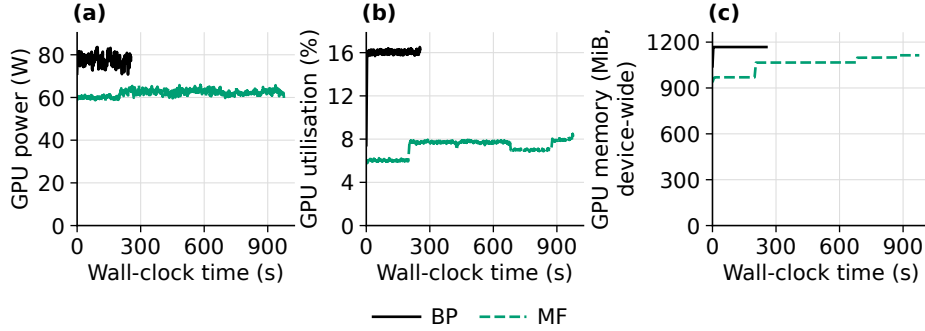


Fig. 1. Hardware monitoring of the BP→MF ladder on the CIFAR-10 3×2000 MLP, median-duration run per rung. All three traces are NVML at 5 Hz (rolling mean over 5 s); none is a Weights & Biases system panel. (a) MF draws roughly 61 W against BP’s 78 W, but for almost four times as long, so the area under the curve—the energy—is far larger. (b) Utilisation and (c) device memory expose MF’s per-layer stages as steps. The device memory reading includes the fixed CUDA context, which is why the rungs separate by so little; per-process figures are given in Table 2.

Does locality buy anything deep supervision does not? Rung 1 of the ladder, BP-DS, is BP with MF’s auxiliary classifiers attached and nothing detached; comparing it with MF isolates the effect of forward-only training with everything else held fixed. The answer splits by task scale. On the two MNIST-scale 2×1000 MLPs the two are statistically equivalent within the ± 0.25 pp margin: MF leads by 0.11 pp $[-0.01, 0.23]$ on MNIST ($n = 20$) and trails by 0.06 pp $[-0.17, 0.06]$ on Fashion-MNIST ($n = 48$). On the two CIFAR 3×2000 MLPs it is not close: MF leads BP-DS by 5.56 pp $[5.37, 5.74]$ on CIFAR-10 and 6.10 pp $[5.88, 6.31]$ on CIFAR-100, both over 20 seeds. Deep supervision alone is *worse* than plain BP on those two tasks (-1.65 and -1.06 pp), whereas MF is better, so the detachment is contributing accuracy rather than merely being tolerated. We report both halves: on small tasks MF’s accuracy is attributable to auxiliary supervision, and on harder ones it is not.

The time–memory frontier is a structural property. Caching activations is the obvious remedy for the recomputation that locality forces, and it does not work. Caching on the device buys no time (1.00 to $1.02\times$) and costs about 40% more per-process memory; caching on the host is slower still ($0.97\times$) and adds 261 pp of BP’s energy on MNIST and 441 pp on Fashion-MNIST on top of MF’s own overhead. Neither strategy reaches a time–memory point that MF without caching does not already dominate. The frontier is not merely an evaluation artefact, because which methods may appear on it at all is decided by the algorithm: BP, BP-DS and MF-Joint cannot cache, since a joint gradient makes any cached activation stale within one optimiser step, whereas MF can, because detachment plus layer-sequential training freeze every preceding layer for the whole of a stage.

The trade-off axis exists only for the local method. That it turns out to be a bad trade is the empirical part; that it is available at all is the algorithmic part.

Is this simply an unoptimised implementation? The natural objection is that BP looks good because a decade of kernel engineering has gone into it. We tested that rather than arguing it. MF’s layer-sequential schedule recomputes the activations of every preceding layer at each stage, an $O(L^2)$ cost a mature implementation would remove by caching, so we implemented the cache and checked it against the recomputing path: over 1200 and 3600 optimiser steps the maximum absolute difference in both loss and weights was exactly 0, so the optimisation is bit-exact. It bought no speedup (1.00 to $1.02\times$ on device, $0.97\times$ on host). The missing optimisation was not missing; it simply does not pay, because at these layer widths the device never exceeds $\approx 38\%$ mean utilisation and the runs are bound by kernel launch latency rather than arithmetic. The same reading covers the other two algorithms: per data pass CaFo-Rand is *cheaper* than BP and FF only 1.5 to $2.0\times$ more expensive, so the end-to-end gaps are made almost entirely of extra epochs. Library maturity is not what separates these methods from BP; convergence rate is.

Intervals, not verdicts. Of 28 accuracy contrasts, eleven are equivalent within the preregistered margin, sixteen are different, and one is inconclusive (Figure 3): CaFo-DFA against BP on the Fashion-MNIST CNN, whose interval spans $[-0.03, 0.61]$ pp at five seeds — wide enough to contain both the margin and values well outside it. We mark such a contrast inconclusive rather than as evidence of parity.

5 Discussion

We interpret the empirical results, discuss methodological limitations, and draw implications for future work. Where interpretations go beyond the data, we label them explicitly as hypotheses.

Why MF is accurate: a regularisation hypothesis. FF’s inefficiency traces to its indirect goodness signal and contrastive data generation, which cost it 2.7 to $3.8\times$ more epochs than BP; CaFo’s DFA pretraining fixes the feature-quality problem at an unaffordable price. A direct supervisory signal appears necessary, and MF’s local projections supply one with minimal overhead — a mechanism close to the local auxiliary losses of [17] and [5], the latter showing in the infinite-width limit that greedy layer-wise training can reduce the effective complexity of the learned representation. This motivates the *regularisation hypothesis*: that greedy layer-wise optimisation implicitly constrains each layer’s representational capacity and so generalises better than joint optimisation. The ladder sharpens it. Auxiliary supervision alone reproduces MF’s accuracy on the MNIST-scale tasks but *loses* 1.06 to 1.65 pp against plain BP on the CIFAR tasks, exactly where MF gains 3.90 to 5.04 pp, so the credit belongs to the detachment rather than to the heads,

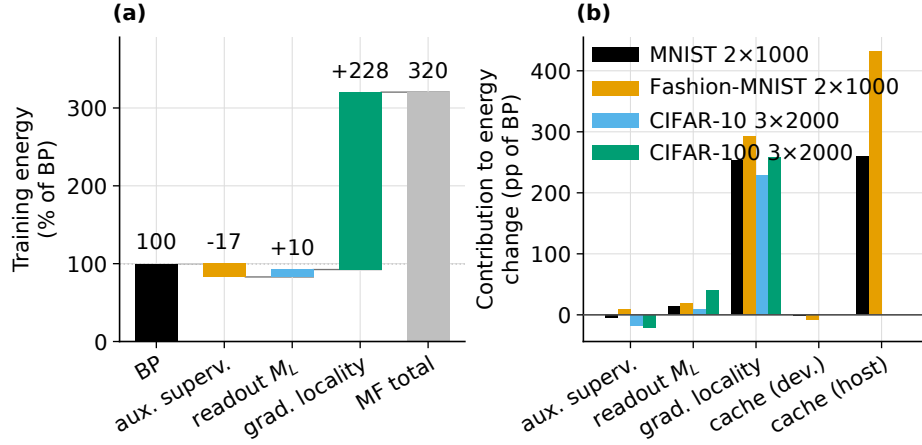


Fig. 2. Decomposition of the BP→MF energy change into five single-change steps. (a) Cumulative GPU energy on the CIFAR-10 3×2000 MLP as a percentage of BP, each bar one rung transition. (b) The same per-step contributions for all four configurations. Gradient locality accounts for essentially the whole increase everywhere. Energy is the trapezoidal integral of the NVML power trace (5 Hz); bars are seed means over 20 to 122 seeds per rung.

and only once the task is hard enough. It remains a hypothesis; testing it needs diagnostics we have not run, such as the train–validation gap over training or sharpness measures.

Memory efficiency depends on which instrument you read. Measured per process with `torch.cuda.max_memory_allocated`, MF holds 27% less than BP on the CIFAR-10 3×2000 MLP and 25% less on CIFAR-100 — far more than the ≈5% the submitted version reported, which came from the device-wide NVML counter in which a fixed ≈800 MiB CUDA context dilutes every difference. **CaFo-Rand** saves 6 to 7% on the same basis, while **FF** and **CaFo-DFA** consume *more* memory than their BP baselines: dual data streams, feedback tensors and projection matrices are real allocations that offset the activation graph one avoids storing. Report the per-process figure, and do not treat “no backward pass” as a memory argument without measuring it.

Implications for efficient training and hardware co-design. The efficiency case for forward-only training does not survive a harmonised stopping rule on this platform, and the traces say why. No configuration we measured exceeds ≈38% mean GPU utilisation, and MF reaches only 7.3% against BP’s 16.2% on CIFAR-10: the layer-sequential schedule issues many small kernels and waits, so the device idles while the run stays resident and the fixed power floor is integrated over four times the duration. This is a launch-latency problem rather than an arithmetic one, which is also why a bit-exact activation cache removed the $O(L^2)$ recomputation without saving any time. That diagnosis is what makes the result

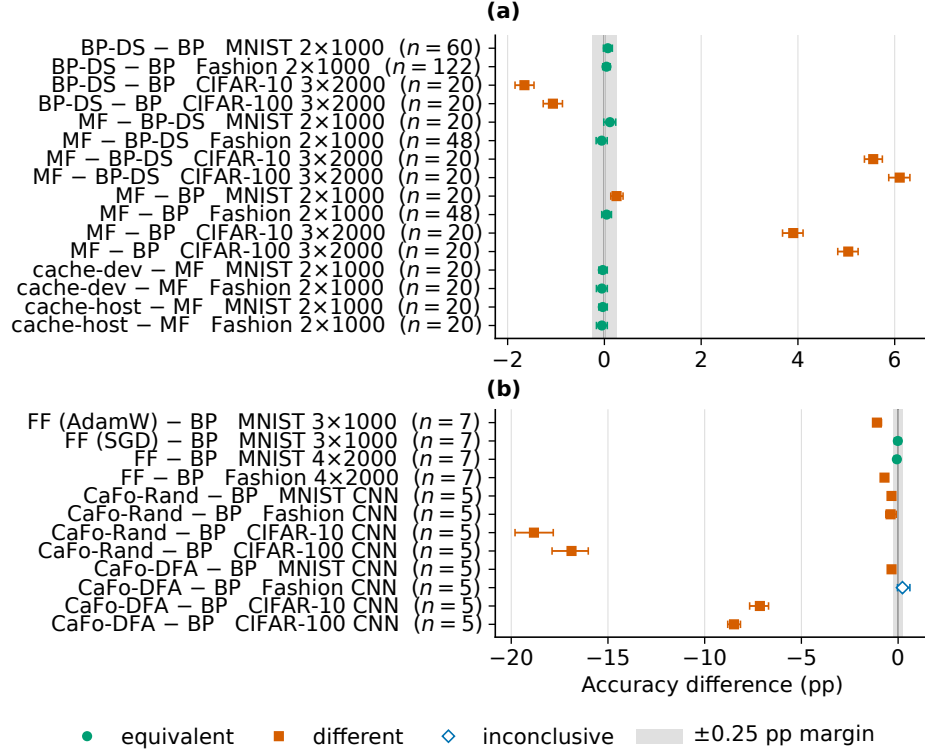


Fig. 3. Paired differences in final test accuracy, bootstrap 95% CIs (10 000 resamples, fixed seed). The shaded band is the preregistered ± 0.25 pp equivalence margin. A contrast is resolved as equivalent only when its whole interval lies inside the band, and as different only when the interval excludes zero; everything else is marked inconclusive. Paired seed counts are given per row. (a) The BP→MF ladder. (b) FF and CaFo against their matched BP baselines.

more than negative: layer-local updates are exactly the work that a pipelined or memory-local execution model need not serialise, and the per-process memory reduction is structural. MF is thus an accuracy and memory result on current hardware and a conditional energy result on hardware that does not yet exist; extrapolating to a sustainability claim would require the advantage at the scale where training energy is societally significant [19, 15].

5.1 Limitations

1. **Architecture and scale.** Results apply to 2–4-layer MLPs (FF, MF) and a 3-block CNN (CaFo). Deeper CNNs, residual networks and transformers are untested, as is behaviour under increasing depth and width, though prior work on local-loss methods [17] suggests scalability.

2. **Datasets.** The four benchmarks are well established but small, and the MLP experiments flatten CIFAR inputs to 1-D vectors following the MF protocol, which destroys spatial structure and caps absolute accuracy far below CNN state-of-the-art. The comparisons are internally valid but say nothing about either algorithm in a competitive vision pipeline.
3. **Hardware specificity.** Every efficiency figure comes from one NVIDIA A100-SXM4-40GB. Since our central diagnosis is that these workloads are launch-latency-bound rather than compute-bound, the results are especially likely to move on hardware with different kernel-launch and memory-bandwidth characteristics, so we regard the direction of the energy result as platform-conditional.
4. **Statistical resolution.** Seed counts are now 5 to 122 per contrast rather than three, but resolution is still bounded. The pooled seed-level SD of test accuracy is 0.284 pp, rising to 0.43 pp on the 3×2000 MLPs, giving a bootstrap CI half-width near ± 0.35 pp at $n = 5$ and ± 0.26 pp at $n = 7$. Against a ± 0.25 pp margin, a contrast below about 0.3 pp cannot be resolved either way at those counts, which is why one is reported inconclusive. The ladder contrasts use 20 seeds or more.
5. **Energy accounting is GPU-only.** NVML reports GPU package power; host CPU, DRAM and PCIe energy are not instrumented, so any variant that moves work off the accelerator is flattered. This matters most for the host-cache MF strategy, whose true total cost is understated by an unknown amount — we still report it as dominated, since it already loses on the instrumented axis. CO₂e estimates rest on a grid-intensity model we do not treat as a measurement and are reported only in the repository.
6. **Hyperparameter search confound.** Each algorithm was tuned independently under an equal trial budget, so differences may partly reflect better hyperparameter discovery. The search cost is excluded from all reported figures; for a slow algorithm such as FF it is substantial and belongs in any total-cost assessment.
7. **Scope.** The contribution is evaluative — the framework, the protocols, the ladder and the measurements. The algorithms are previously published [4, 22, 3].

6 Conclusion

We presented a hardware-validated benchmarking study of three forward-only, BP-free training algorithms — FF, CaFo and MF — against fairly tuned BP baselines, under one fairness framework, on MLP and CNN architectures across four vision benchmarks.

On its native MLP architectures MF matched or surpassed BP accuracy in every configuration, by 0.01 to 5.04 pp, and held up to 27% less per-process device memory. It was not cheaper to train. Under a stopping rule harmonised across algorithms, MF is $4.0\times$ slower and draws $3.2\times$ the energy of BP on CIFAR-10; the 41% energy saving reported in the accepted version is reproducible only under

the submitted protocol, in which each algorithm was allowed a different amount of training before it could stop, and its sign reverses once that is equalised. FF and CaFo are more expensive still on every axis but memory. An ablation ladder attributes the increase almost entirely to gradient locality, worth 228 to 272 pp of BP’s energy, and shows that a deep-supervision BP control reproduces MF’s accuracy on the MNIST-scale tasks but falls more than 5 pp short of it on CIFAR.

These results are specific to small architectures and benchmarks on a single GPU, and their efficiency half is conditional on a platform where nothing we measured exceeded $\approx 38\%$ mean GPU utilisation. Within that scope the work provides a reproducible framework, an attribution of cost to a specific architectural property rather than to an algorithm as a whole, and a corrected empirical basis — including corrections to our own earlier claims — for evaluating BP-free methods.

Acknowledgments. This work was supported by the Ministry of Science and Higher Education of Poland and by the National Science Centre (NCN), Poland, under grant OPUS-29, DEC-2025/57/B/ST6/04377. We thank the PLGrid Infrastructure and ACK Cyfronet AGH for the computational resources on the Athena cluster, and AGH University of Krakow for institutional support. The research was conducted as part of a master’s thesis by Przemysław Spyra under the supervision of Witold Dzwinel: Przemysław Spyra developed and implemented the methodology, experiments and analysis; Witold Dzwinel provided conceptual guidance, critical feedback and direction.

Code and data. The source code, every configuration file including the preserved pre-revision settings, the Optuna search spaces and selected values, the full environment listing, the NVML instrumentation, the tidy table behind every number in this paper, and the per-run logs are available at <https://github.com/Przemyslaw11/BeyondBackpropagation>.

References

1. Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 2623–2631 (2019)
2. Desislavov, R., Lazovic, D., Zhou, J., Grosso, P.: Measuring the energy consumption and efficiency of deep neural networks: A survey. arXiv preprint arXiv:2403.08151 (2024)
3. Gong, J., Li, B., Abdulla, W.: Mono-Forward: Backpropagation-free algorithm for efficient neural network training harnessing local errors. arXiv preprint arXiv:2501.09238 (2025)
4. Hinton, G.: The forward-forward algorithm: Some preliminary investigations. arXiv preprint arXiv:2212.13345 (2022)
5. Ishikawa, S., Yokota, R., Karakida, R.: Local loss optimization in the infinite width: Stable parameterization of predictive coding networks and target propagation. In: Proceedings of the 13th International Conference on Learning Representations (ICLR) (2025)

6. Jaderberg, M., Czarnecki, W.M., Osindero, S., Vinyals, O., Graves, A., Silver, D., Kavukcuoglu, K.: Decoupled neural interfaces using synthetic gradients. In: Proceedings of the 34th International Conference on Machine Learning (ICML). pp. 1627–1635 (2017)
7. Jouppi, N.P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A., et al.: In-datacenter performance analysis of a tensor processing unit. In: Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA). pp. 1–12 (2017)
8. Krizhevsky, A., Hinton, G.: Learning multiple layers of features from tiny images. Tech. rep., University of Toronto (2009)
9. Krizhevsky, A., Sutskever, I., Hinton, G.E.: ImageNet classification with deep convolutional neural networks. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 25, pp. 1097–1105 (2012)
10. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE **86**(11), 2278–2324 (1998)
11. Lee, C.Y., Xie, S., Gallagher, P., Zhang, Z., Tu, Z.: Deeply-supervised nets. In: Proceedings of the 18th International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 38, pp. 562–570 (2015)
12. Lillicrap, T.P., Santoro, A., Marris, L., Akerman, C.J., Hinton, G.: Backpropagation and the brain. Nature Reviews Neuroscience **21**(6), 335–346 (2020)
13. Lorberbom, G., Gat, I., Adi, Y., Schwing, A., Hazan, T.: Layer collaboration in the forward-forward algorithm. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 38, pp. 14141–14148 (2024)
14. Papachristodoulou, A., Kyrkou, C., Timotheou, S., Theodoridis, T.: Convolutional channel-wise competitive learning for the forward-forward algorithm. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 38, pp. 14536–14544 (2024)
15. Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.M., Rothchild, D., So, D.R., Texier, M., Dean, J.: Carbon emissions and large neural network training. arXiv preprint arXiv:2104.10350 (2021)
16. Prechelt, L.: Early stopping—but when? In: Orr, G., Müller, K.R. (eds.) Neural Networks: Tricks of the Trade, pp. 55–69. Springer (1998)
17. Ren, M., Kornblith, S., Liao, R., Hinton, G.: Scaling forward gradient with local losses. In: Proceedings of the 11th International Conference on Learning Representations (ICLR) (2023)
18. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. Nature **323**(6088), 533–536 (1986)
19. Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for deep learning in NLP. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL). pp. 3645–3650. Association for Computational Linguistics (2019)
20. World Economic Forum, Accenture: Artificial intelligence’s energy paradox: Balancing challenges and opportunities. Ai governance alliance report, World Economic Forum (January 2025)
21. Xiao, H., Rasul, K., Vollgraf, R.: Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747 (2017)
22. Zhao, G., Wang, T., Li, Y., Jin, Y., Lang, C., Ling, H.: The cascaded forward algorithm for neural network training. arXiv preprint arXiv:2303.09728 (2023)